CS3300+CS3310 — Compiler Design (+ lab)

Tier	A
Phase (order)	4
Do this first	CS2200, CS2705, CS2600
Estimated effort	14 weeks, ~9 h/week
Unlocks	CS6848, CS6013, CS6843

Roadmap

1. Lexing: regular expressions to tokens (flex).
2. Parsing: top-down and LALR (bison); resolve conflicts.
3. Syntax-directed translation, semantic analysis, type checking.
4. Intermediate code, runtime environments, simple optimisation and register allocation.

Video courses (all links checked on YouTube on 2026-10-22)

🎓 Main course: follow it start to end

- NOC Jan 2019: Compiler Design (62 videos). NPTEL *Compiler Design* (Santanu Chattopadhyay, IIT Kharagpur, 2019).

🎥 Other full lecture series (pick one if the main one doesn't click, or for topics it skips)

- Computer Science - Principles of Compiler Design (40 videos). NPTEL *Principles of Compiler Design* (Y.N. Srikant, IISc): strongest on optimisation and code generation.
- Compiler Design by Sorav Bansal - IIT Delhi (100+ videos). Sorav Bansal (IIT Delhi) *Compiler Design*: modern, project-driven.
- Stanford CS143: Compilers (19 videos). Stanford CS143 (Alex Aiken) re-upload: the classic COOL-compiler course.

⚡ Short videos for intuition and revision

- Compiler Design (42 videos). Neso Academy *Compiler Design*: parsing tables step by step.

Study plan (week by week)

Lecture numbers are positions in the main playlist above. Click a number to open that video.

Week	Lectures	What to do
1	1–7	Compiler structure and phases. Read the Crafting Interpreters intro; pick your project language (a tiny C subset).

Week	Lectures	What to do
2-3	8–16	Lexical analysis: regex, finite automata, lex/flex. Write a hand-coded lexer, then the same with flex.
4-7	17–39	Parsing: top-down (LL(1), recursive descent), bottom-up (SLR, LR(1), LALR), yacc/bison. Write a recursive-descent parser producing an AST; then one grammar in bison.
8	40–43	Type checking. Add a type checker pass to your AST.
9	44–50	Symbol tables and runtime environments (activation records, stack layout). Implement scoped symbol tables.
10-12	51–62	Intermediate code generation (three-address code). Emit TAC, then generate x86-64 or RISC-V assembly and run real programs. Code optimisation: Sorav Bansal (alt) lectures.

▼ **All lectures in *NOC Jan 2019: Compiler Design* (62 videos)**

1. Compiler Design
2. Lecture : 01 Introduction
3. Lecture 02: Introduction (Contd.)
4. Lecture : 03 Introduction (Contd.)
5. Lecture : 04 Introduction (Contd.)
6. Lecture : 05 Introduction (Contd.)
7. Lecture 06: Introduction (Contd.)
8. Lecture 07: Lexical Analysis
9. Lecture 08: Lexical Analysis (Contd.)
10. Lecture 09: Lexical Analysis (Contd.)
11. Lecture 10: Lexical Analysis (Contd.)
12. Lecture 11: Lexical Analysis (Contd.)
13. Lecture 12: Lexical Analysis (Contd.)
14. Lecture 13: Lexical Analysis (Contd.)
15. Lecture 14: Lexical Analysis (Contd.)
16. Lecture 15: Lexical Analysis (Contd.)
17. Lecture 16: Parser
18. Lecture 17: Parser (Contd.)
19. Lecture 18: Parser (Contd.)
20. Lecture 19: Parser (Contd.)
21. Lecture 20: Parser (Contd.)
22. Lecture 21: Parser (Contd.)
23. Lecture 22: Parser (Contd.)
24. Lecture 23: Parser (Contd.)
25. Lecture 24: Parser (Contd.)
26. Lecture 25: Parser (Contd.)

27. Lecture 26: Parser (Contd.)
28. Lecture 27: Parser (Contd.)
29. Lecture 28: Parser (Contd.)
30. Lecture 29: Parser (Contd.)
31. Lecture 30: Parser (Contd.)
32. Lecture 31: Parser (Contd.)
33. Lecture 32: Parser (Contd.)
34. Lecture 33: Parser (Contd.)
35. Lecture 34: Parser (Contd.)
36. Lecture 35: Parser (Contd.)
37. Lecture 36: Parser (Contd.)
38. Lecture 37: Parser (Contd.)
39. Lecture 38: Parser (Contd.)
40. Lecture 39: Type Checking
41. Lecture 40: Type Checking(Contd.)
42. Lecture 41: Type Checking(Contd.)
43. Lecture 42: Type Checking(Contd.)
44. Lecture 43: Symbol Table
45. Lecture 44: Symbol Table (Contd.)
46. Lecture 45: Symbol Table (Contd.)
47. Lecture 46: Symbol Table (Contd.) and Runtime Environment
48. Lecture 47: Runtime Environment
49. Lecture 48: Runtime Environment (Contd.)
50. Lecture 49: Runtime Environment (Contd.)
51. Lecture 50: Intermediate Code Generation
52. Lecture 51: Intermediate Code Generation (Contd.)
53. Lecture 52: Intermediate Code Generation (Contd.)
54. Lecture 53: Intermediate Code Generation (Contd.)
55. Lecture 54: Intermediate Code Generation (Contd.)
56. Lecture 55: Intermediate Code Generation (Contd.)
57. Lecture 56: Intermediate Code Generation (Contd.)
58. Lecture 57: Intermediate Code Generation (Contd.)
59. Lecture 58: Intermediate Code Generation (Contd.)
60. Lecture 59: Intermediate Code Generation (Contd.)
61. Lecture 60: Intermediate Code Generation (Contd.)
62. Lecture 61: Intermediate Code Generation (Contd.)

Books, notes and websites

- Book: Aho, Lam, Sethi, Ullman, *Compilers: Principles, Techniques, and Tools* (the course text)

- ✅ NPTEL *Compiler Design* (Santanu Chattopadhyay, IIT Kharagpur) — onlinecourses.nptel.ac.in/noc22_cs14/preview
- ✅ Prof. Krishna Nandivada's own CS3300 slides — cse.iitm.ac.in/~krishna/cs3300
- *Crafting Interpreters* — craftinginterpreters.com (free)
- Stanford CS143 *Compilers* (online)

Practice / proof of understanding

Build a compiler for a small language (variables, ints, if/while, functions) that emits assembly or LLVM IR.

What your IITM course sheet says

From the PDF: CS3300

Official description: The objectives are to teach students a understanding of some of the standard passes in a general purpose compiler and to provide hands-on experience in implementing a compiler for a subset of a high-level language such as Java.

Official syllabus (trimmed):

> Introduction to language translators and overview of the compilation process. Lexical analysis: specification of tokens, token recognition, conflict resolution Parsing: Overview of CFG, Parse trees and derivations, left recursion, left factoring, top-down parsing, LALR parsing, conflict resolution, dangling-else. Syntax directed translation. Semantic analysis, Type checking, intermediate code generation. Runtime environments: activation records, heap management Code optimization: basic blocks, liveness, register allocation. Advanced topics: Overview of machine dependent and independent optimizations.

Official textbooks: 1. Alfred V. Aho, Monica Lam, Ravi Sethi and Jeffrey D. Ullman, "Compilers: Principles, Techniques, and Tools", second edition, Pearson Education, 2007 2. Modern compiler implementation in Java, Second Edition, Andrew W. Appel, Jens Palsberg, Cambridge University Press, 2002.

Official references: 1. Computer Organization and Design: The Hardware/Software Interface, David Patterson and John Hennessy, Morgan Kaufmann, 1998 2. The Java programming language, Ken Arnold, James Gosling and David Holmes, Addison-Wesley, 2000.

From the PDF: CS3310

Official description: A language subset will be defined and used during the lab course. The programming exercises here consist of implementing the basic components of a compiler. The constructs in this subset are found in most programming languages. They allow programs such as recursive and non-recursive sorting, and matrix multiplication to be expressed.

Official syllabus (trimmed):

> Lexing, Parsing, type checking, intermediate code generation, register allocation, simple optimizations, code generation.

Official textbooks: NIL

Official references: 1. Modern compiler implementation in Java, Second Edition, Andrew W. Appel, Jens Palsberg, Cambridge University Press, 2002. 2. A.T. Schreiner and H.G. Friedman, Jr., "Introduction to

Compiler construction with UNIX", Prentice Hall, 1985.